SAÉ 2.03 : Rapport Technique Intermédiaire

Table des matières

1. Préparation de la machine virtuelle . . . 1
   1.1. Création et caractéristiques de la VM . . . 2
   1.2. Installation de l'OS de base . . . 2
   1.3. Préparation du système et Suppléments invités . . . 3
2. Réponses aux questions techniques (Semaine 07) . . . 3
   2.1. Configuration matérielle dans VirtualBox (Question 1) . . . 3
   2.2. Installation OS de base (Question 2) . . . 4
   2.3. Sudo et Suppléments invités (Questions 3 & 4) . . . 5
3. À propos de la distribution Debian (Question 4.2) . . . 5
4. Installation préconfigurée (Question 5) . . . 6
   4.1. Ajustement de la pré-configuration . . . 6
   4.2. Notre configuration automatisée . . . 6
5. Étude des applications clientes (Semaine 10) . . . 7
   5.1. Configuration globale de Git . . . 7
   5.2. Concepts de base (Question 1) . . . 7
   5.3. Protocoles réseau pour Git (Question 2) . . . 8
   5.4. Manipulation pratique de Git (Question 3) . . . 8
6. Étude des applications clientes (Semaine 10) . . . 11
   6.1. Configuration globale de Git . . . 11
   6.2. Concepts de base (Question 1) . . . 11
   6.3. Protocoles réseau pour Git (Question 2) . . . 11
   6.4. Manipulation pratique de Git (Question 3) . . . 12

Présentation

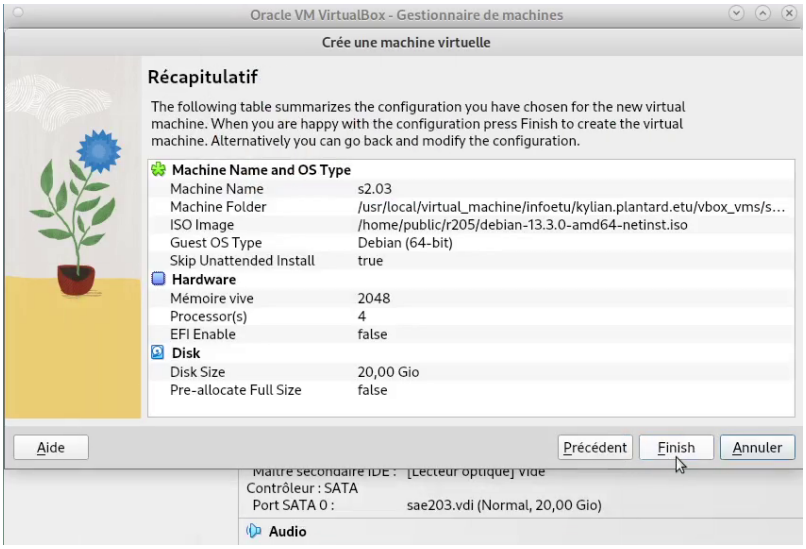
Ce rapport retrace l'installation, la configuration et l'automatisation d'une machine virtuelle sous Debian 13, ainsi que les réponses aux questions techniques associées.

1. Préparation de la machine virtuelle

1.1. Création et caractéristiques de la VM

Nous avons mis en place une machine virtuelle dans VirtualBox en respectant les spécifications suivantes :

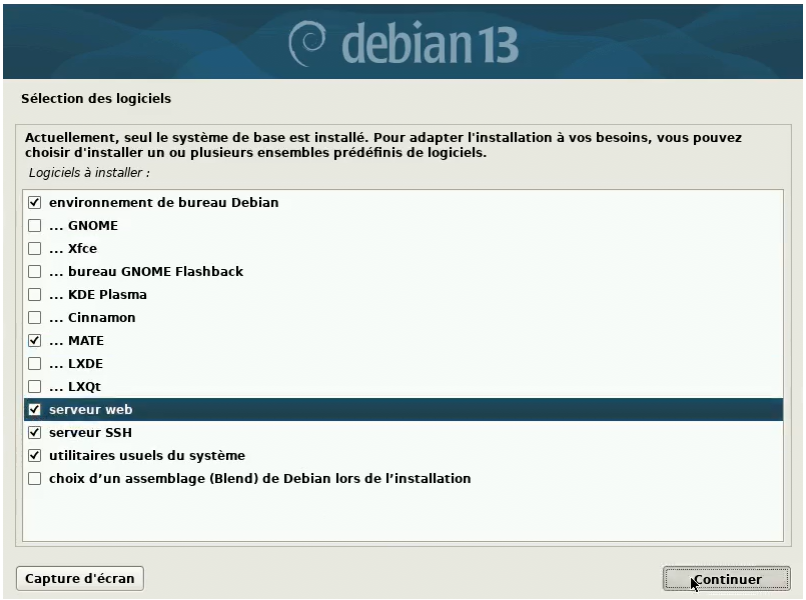
- **Nom de la machine** : `s2.03`
- **Type de système** : Linux
- **Version** : Debian (64-bit)
- **Mémoire vive (RAM)** : 2048 Mo
- **Disque dur** : 20 Go (une seule partition de la taille totale)



1.2. Installation de l'OS de base

Lors de l'installation du système d'exploitation à partir de l'image ISO, nous avons appliqué les paramètres suivants :

1. **Nom de la machine** : `serveur`
2. **Miroir réseau** : `http://debian.polytech-lille.fr`
3. **Comptes créés** :
   - Administrateur : `root`
   - Utilisateur standard : `user`
4. **Sélection des logiciels** :
   - Environnement de bureau Debian
   - MATE
   - Serveur web
   - Serveur SSH
   - Utilitaires usuels du système



1.3. Préparation du système et Suppléments invités

Afin de faciliter l'administration du système, nous avons accordé les droits `sudo` à l'utilisateur `user` via la console tty1 (`Ctrl` + `Alt` + `F1`) en étant connecté en tant que `root` :

```
usermod -aG sudo user
```

man 8 usermod [1]

Nous avons ensuite inséré le CD virtuel des **Additions Invité** (Guest Additions) depuis le menu de VirtualBox, puis nous l'avons monté et installé en ligne de commande :

```
sudo mount /dev/cdrom /mnt
sudo /mnt/VBoxLinuxAdditions.run
```

man 8 mount [2]

Un redémarrage a permis d'activer le redimensionnement dynamique de la fenêtre.

2. Réponses aux questions techniques (Semaine 07)

2.1. Configuration matérielle dans VirtualBox (Question 1)

Question	Réponse justifiée
Que signifie "64-bit" ?	Cela désigne l'architecture du processeur capable de traiter des données par blocs de 64 bits [3]. C'est l'architecture la plus courante pour les systèmes modernes.
Configuration réseau par défaut ?	Par défaut, VirtualBox émule un adaptateur réseau ethernet `Intel PRO/1000 MT Desktop` avec un cable virtuel connecté.
Pourquoi 2048 Mo de RAM ? Et avec 512 Mo ?	2048 Mo permettent de faire tourner confortablement l'installation ainsi que l'environnement graphique MATE. Avec seulement 512 Mo, le système utiliserait massivement la partition d'échange (Swap), rendant la machine extrêmement lente, et l'interface graphique pourrait même refuser de se lancer par manque de mémoire.
Mode réseau par défaut ?	Le mode NAT. Il isole la VM tout en lui donnant accès à Internet via la connexion de la machine hôte.
Adresse IP de la VM ?	`ip address` [4] L'adresse IP est `10.0.2.15`.
Adresse de la passerelle ?	`ip route` La passerelle par défaut est `10.0.2.2`. Elle correspond au routeur virtuel créé par VirtualBox pour le mode NAT.

2.2. Installation OS de base (Question 2)

Question	Réponse justifiée
Fichier iso bootable ?	C'est une image disque (une copie exacte d'un CD/DVD) contenant les fichiers nécessaires pour amorcer (booter) un ordinateur et installer un système d'exploitation.
MATE et GNOME ?	Ce sont des Environnements de Bureau (Desktop Environments) pour Linux. Ils fournissent l'interface graphique (fenêtres, icônes, barres des tâches). GNOME est plus lourd et moderne, MATE est un dérivé plus léger.
Serveur mandataire (proxy) ?	C'est un serveur qui agit comme intermédiaire entre les requêtes d'un client (notre VM) et un autre serveur (Internet). Il peut filtrer, mettre en cache ou sécuriser les connexions.
Serveur web ?	Un logiciel (comme Apache ou Nginx) qui répond aux requêtes HTTP/HTTPS pour distribuer des pages web à des clients (navigateurs).
Serveur SSH installé ?	Le paquet installé est `openssh-server`. On vérifie son état avec la commande `systemctl status ssh`.

Question	Réponse justifiée
Test de connexion SSH ?	Commande : `ssh user@10.0.2.15`. **Problème rencontré** : La connexion échoue ("Timeout"). **Pourquoi ?** Parce que le mode réseau NAT de VirtualBox crée un réseau séparé de celui de l'hôte. Il faut configurer une redirection de ports (Port Forwarding) dans VirtualBox pour que cela fonctionne.

2.3. Sudo et Suppléments invités (Questions 3 & 4)

- **Appartenance aux groupes** : Pour savoir à quels groupes appartient l'utilisateur `user`, on utilise la commande `groups user` [5] ou `id user` [6].
- **Différence entre `su` et `sudo`** :
  - `su` [7] (substitute user) permet de changer d'utilisateur (souvent pour devenir `root`) et requiert le mot de passe de l'utilisateur cible.
  - `sudo` [8] (substitute user do) permet d'exécuter une seule commande avec les privilèges d'un autre utilisateur (généralement `root`), mais nécessite le mot de passe de l'utilisateur courant.
- **Version du noyau Linux** : `uname -r` [9] : `6.12.73+deb13-amd64`
- **Utilité des suppléments invités** : Ils optimisent la VM. Deux raisons de les installer :
  - Permettre le redimensionnement dynamique de la fenêtre (ajustement automatique de la résolution).
  - Activer le presse-papiers partagé entre la machine hôte et la VM.
- **Commande `mount`** : En général, elle sert à attacher un système de fichiers (disque, clé USB) à l'arborescence du système (point de montage). Dans notre cas, elle a servi à rendre accessible le contenu du lecteur CD-ROM virtuel pour y exécuter le script d'installation.

3. À propos de la distribution Debian (Question 4.2)

Pour répondre à ces questions, nous avons consulté la documentation officielle de Debian.

1. **Le Projet Debian et son nom** : C'est une association d'individus qui ont pour cause commune de créer un système d'exploitation libre. Le nom "Debian" a été imaginé par son créateur, Ian Murdock, en combinant le prénom de sa petite amie de l'époque (Debra) et le sien (Ian).
   What is the Debian Project?
2. **Durées de prise en charge** :
   - *Minimale (Standard)* : Environ 3 ans (1 an après la sortie de la version stable suivante).
   - *LTS (Long Term Support)* : 5 ans au total. Debian LTS
   - *ELTS (Extended LTS)* : Jusqu'à 10 ans (géré par une organisation tierce, Freexian). Debian ELTS

3. **Mises à jour de sécurité** : L'équipe de sécurité Debian fournit un support pour la version stable pendant environ 1 an après la sortie d'une nouvelle version stable. Debian Release Lifecycle
4. **Versions activement maintenues** : Il y en a au minimum 3. Leurs noms génériques sont : *Stable*, *Testing* (en test), et *Unstable* (instable, surnommée *Sid*). On peut aussi compter *Oldstable*. Debian Release Lifecycle
5. **Origine des noms de code** : Tous les noms de code des versions Debian proviennent des personnages des films d'animation *Toy Story* (ex. : Bullseye, Bookworm, Trixie). Toy Story - Debian Wiki
6. **Architectures de Trixie (Debian 13)** : La version Trixie de Debian supporte: *amd64*, *aarch64*, *armhf*, *powerpc*, *riscv64* et *system z*. Debian Trixie Release Information
7. **Premier nom de code** : `Buzz` (Debian 1.1), annoncé le 17 juin 1996. Debian Releases
8. **Dernier nom de code annoncé** : La version 14 est `Forky` et la version 15 sera `Duke`. Annonce de Forky - Debian Mailing List

4. Installation préconfigurée (Question 5)

Attention
La modification de l'installation automatisée se fait exclusivement dans le fichier `preseed-fr.cfg`.

4.1. Ajustement de la pré-configuration

Question	Réponse
Différence `pkgsel/include` et `tasksel`	`d-i pkgsel/include` permet de spécifier une liste de **paquets individuels** précis à installer (ex. : `git`, `curl`). `tasksel tasksel/first` installe des **tâches entières** qui regroupent des centaines de paquets (ex. : `mate-desktop` ou `web-server`).
Sécurité du preseed	Oui, le fichier preseed contient les mots de passe de `root` et de `user` en clair. En production, il faut protéger ce fichier en le stockant sur un serveur web interne sécurisé (HTTPS avec authentification), en restreignant drastiquement les droits de lecture du fichier physique ou en écrivant le hash uniquement.

4.2. Notre configuration automatisée

Pour répondre aux consignes de l'installation automatisée, nous avons édité le fichier `preseed-fr.cfg`. Voici les modifications clés apportées :

Installation de MATE :

```
d-i tasksel/first multiselect standard, mate-desktop
```

Ajout des paquets supplémentaires :

```
d-i pkgsel/include string sudo git sqlite3 curl bash-completion fastfetch
```

Ajout de l'utilisateur user dans root : Nous avons modifié le paramètre par défaut des groupe du nouvel utilisateur pour y rajouter `sudo` :

```
-d-i passwd/user-default-groups string audio cdrom video
+d-i passwd/user-default-groups string audio cdrom video sudo
```

ajouter conclusion de la semaine 1

#semain10

5. Étude des applications clientes (Semaine 10)

5.1. Configuration globale de Git

La commande `git config --global` [10] permet de définir des variables globales pour l'ordinateur, qui seront utilisées lors de vos commits (comme votre nom ou votre e-mail).
Elles ne sont évidemment à faire qu'une fois par machine, et pas à chaque projet. Si l'email est différent que celui du compte, les commits ne seront pas associés au profil.

5.2. Concepts de base (Question 1)

Question	Réponse justifiée
Différence entre Git et les logiciels comme GitHub/GitLab/Forgejo ?	Git est l'outil de contrôle de version local en ligne de commande, alors que GitHub/GitLab/Forgejo sont des forges logicielles collaboratives (des plateformes web) permettant d'héberger et de gérer les dépôts Git en ligne.
Qu'est-ce qu'un dépôt Git ? Où sont stockées les données d'un dépôt local ?	Un dépôt Git, souvent appelé « repo », est un emplacement de stockage pour les fichiers d'un projet et l'ensemble de l'historique de leurs modifications. Ces données sont stockées de façon masquée dans le dossier local `.git`.
Différence entre un commit, une branche et un tag ?	Un **commit** est un instantané contenant les différences de chaque fichier par rapport au commit précédent. Une **branche** est une ligne de développement indépendante (une suite de commits). Un **tag** est une étiquette figée collée sur un commit précis (souvent utilisée pour marquer les versions logicielles).

Question	Réponse justifiée
Qu'est-ce qu'un dépôt distant (remote) ? Comment lister les remotes configurés ?	Un dépôt distant est un dépôt accessible par le réseau (via Internet ou un réseau local d'entreprise). La commande `git remote -v` permet de lister ceux qui sont configurés sur le projet.
Différence entre git pull et git fetch ?	`git fetch` : récupère les données du dépôt distant vers le dépôt local (sans modifier vos fichiers de travail en cours). `git pull` : effectue un **fetch** puis fusionne directement ces nouvelles données avec la branche locale actuelle.

5.3. Protocoles réseau pour Git (Question 2)

Question	Réponse justifiée
Quels sont les protocoles supportés par Git pour accéder à un dépôt ?	• Protocole Local (accès au système de fichiers sans port) • Protocole Git • Protocole HTTP/HTTPS (mode intelligent et idiot) [11] • Protocole SSH [12]
Sur quels ports réseau fonctionnent ces protocoles ?	Le protocole Git fonctionne sur le port `9418`, HTTP sur le port `80`, HTTPS sur `443` et SSH sur le port `22`.
Comment configurer l'authentification SSH pour Git ?	Il faut générer une paire de clés SSH sur l'ordinateur avec la commande `ssh-keygen -t ed25519 -C "exemple@univ-lille.fr"`, puis copier le contenu de la clé publique générée pour l'ajouter dans les paramètres de sécurité du compte sur la forge logicielle.

5.4. Manipulation pratique de Git (Question 3)

Question	Réponse justifiée / Commandes
Créez un nouveau dépôt Git local. Quelle commande utilisez-vous ? Que se passe-t-il ?	La commande `git init` crée un sous-dossier `.git` qui initialise la structure d'un dépôt vide dans le répertoire actuel.

Question	Réponse justifiée / Commandes

Ajoutez plusieurs fichiers, faites au moins 3 commits. Utilisez git log pour visualiser l'historique.	
	`touch fichier1 fichier2 fichier3 fichier4`
	`git add fichier1`
	`git commit -m "ajout du premier fichier"`
	`git add fichier2`
	`git commit -m "ajout du second fichier"`
	`git add fichier3 fichier4`
	`git commit -m "ajout du fichier 3 et 4"`
	`git log`
	
	`user@S203:~/exemple$ git log`
	`commit be99f4166cf068d32233745e234c032dd82cc12b (HEAD -> master)`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:37 2026 +0100`
	
	`ajout du fichier 3 et 4`
	
	`commit 1cb9411aefaa8fd404589caa304307193154490e`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:10 2026 +0100`
	
	`ajout du second fichier`
	
	`commit ad4e99ad7d5e6aa8dc48a2c5df1662eeec81be1b`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:52:50 2026 +0100`
	
	`ajout du premier fichier`
	
	Le résultat est une liste des commits du plus récent au plus ancien avec leurs identifiants (be99f4166…), l'auteur, la date et le message de commit.

Créez une nouvelle branche, faites des modifications, puis fusionnez (merge) cette branche avec la principale. Expliquez.	`user@S203:~/exemple$ git checkout -b exemple`
	`Basculement sur la nouvelle branche 'exemple'`
	`user@S203:~/exemple$ echo "dans une branche" > texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit -m "ajout du texte"`
	`[exemple 1ac7443] ajout du texte`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	`user@S203:~/exemple$ git checkout master`
	`Basculement sur la branche 'master'`
	`user@S203:~/exemple$ git merge exemple`
	`Mise à jour be99f41..1ac7443`
	`Fast-forward`
	`texte | 1 +`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	
	Une branche est une chaine de commits parallèle aux autres branches (ici `master` et `exemple`).
	La fusion (merge) permet de combiner les changements d'une branche dans une autre en rajoutant les commits de la branche source vers la branche cible.

Question	Réponse justifiée / Commandes

Qu'est-ce qu'un conflit de fusion ? Provoquez-en un et montrez comment le résoudre.	`user@S203:~/exemple$ git merge conflit`
	`Fusion automatique de texte`
	`CONFLIT (contenu) : Conflit de fusion dans texte`
	`La fusion automatique a échoué ; réglez les conflits et validez le résultat.`
	`user@S203:~/exemple$ nano texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit`
	`[master 8591824] Merge branch 'conflit'`
	
	Un conflit survient lorsque deux branches modifient le même fichier de façon différente.
	Lors de la fusion, Git met le processus en pause et demande une résolution manuelle (choisir le code à garder et supprimer les marqueurs de conflit).
	Pour résoudre le conflit, on modifie le fichier puis on le `git add` pour le marquer comme résolu, avant de faire un commit de fusion.
	
	`<<<<<<< HEAD`
	`conflit 1`
	`=======`
	`conflit 2`
	`>>>>>>> conflit`

Utilisez git diff pour comparer deux commits. Expliquez la sortie.	`diff --git a/texte b/texte`
	`index 9d600a1..7255def 100644`
	`--- a/texte`
	`+++ b/texte`
	`@@ -1 +1 @@`
	`-dans une branche`
	`+conflit 1`
	
	Les premières lignes indiquent les fichiers comparés (`a/texte` et `b/texte`) et les méta-données de la comparaison. (`index 9d600a1..7255def`)
	Les lignes précédées d'un `-` en rouge sont des suppressions et celles avec un `+` en vert sont des ajouts.
	Les autres lignes affichées servent de contexte autour du code modifié.

Qu'est-ce que le fichier .gitignore ? Créez-en un pour ignorer les fichiers Java (.class, .jar).	C'est un fichier texte permettant de lister les motifs des fichiers/dossiers qui ne doivent pas être suivis et versionnés par Git.
	
	`*.class`
	`*.jar`

#semain10

6. Étude des applications clientes (Semaine 10)

6.1. Configuration globale de Git

La commande `git config --global` [13] permet de définir des variables globales pour l'ordinateur, qui seront utilisées lors de vos commits (comme votre nom ou votre e-mail).
Elles ne sont évidemment à faire qu'une fois par machine, et pas à chaque projet. Si l'email est différent que celui du compte, les commits ne seront pas associés au profil.

6.2. Concepts de base (Question 1)

Question	Réponse justifiée
Différence entre Git et les logiciels comme GitHub/GitLab/Forgejo ?	Git est l'outil de contrôle de version local en ligne de commande, alors que GitHub/GitLab/Forgejo sont des forges logicielles collaboratives (des plateformes web) permettant d'héberger et de gérer les dépôts Git en ligne.
Qu'est-ce qu'un dépôt Git ? Où sont stockées les données d'un dépôt local ?	Un dépôt Git, souvent appelé « repo », est un emplacement de stockage pour les fichiers d'un projet et l'ensemble de l'historique de leurs modifications. Ces données sont stockées de façon masquée dans le dossier local `.git`.
Différence entre un commit, une branche et un tag ?	Un **commit** est un instantané contenant les différences de chaque fichier par rapport au commit précédent. Une **branche** est une ligne de développement indépendante (une suite de commits). Un **tag** est une étiquette figée collée sur un commit précis (souvent utilisée pour marquer les versions logicielles).
Qu'est-ce qu'un dépôt distant (remote) ? Comment lister les remotes configurés ?	Un dépôt distant est un dépôt accessible par le réseau (via Internet ou un réseau local d'entreprise). La commande `git remote -v` permet de lister ceux qui sont configurés sur le projet.
Différence entre git pull et git fetch ?	`git fetch` : récupère les données du dépôt distant vers le dépôt local (sans modifier vos fichiers de travail en cours). `git pull` : effectue un **fetch** puis fusionne directement ces nouvelles données avec la branche locale actuelle.

6.3. Protocoles réseau pour Git (Question 2)

Question	Réponse justifiée
Quels sont les protocoles supportés par Git pour accéder à un dépôt ?	• Protocole Local (accès au système de fichiers sans port) • Protocole Git • Protocole HTTP/HTTPS (mode intelligent et idiot) [14] • Protocole SSH [15]
Sur quels ports réseau fonctionnent ces protocoles ?	Le protocole Git fonctionne sur le port `9418`, HTTP sur le port `80`, HTTPS sur `443` et SSH sur le port `22`.
Comment configurer l'authentification SSH pour Git ?	Il faut générer une paire de clés SSH sur l'ordinateur avec la commande `ssh-keygen -t ed25519 -C "exemple@univ-lille.fr"`, puis copier le contenu de la clé publique générée pour l'ajouter dans les paramètres de sécurité du compte sur la forge logicielle.

6.4. Manipulation pratique de Git (Question 3)

Question	Réponse justifiée / Commandes
Créez un nouveau dépôt Git local. Quelle commande utilisez-vous ? Que se passe-t-il ?	La commande `git init` crée un sous-dossier `.git` qui initialise la structure d'un dépôt vide dans le répertoire actuel.

Question	Réponse justifiée / Commandes

Ajoutez plusieurs fichiers, faites au moins 3 commits. Utilisez git log pour visualiser l'historique.	
	`touch fichier1 fichier2 fichier3 fichier4`
	`git add fichier1`
	`git commit -m "ajout du premier fichier"`
	`git add fichier2`
	`git commit -m "ajout du second fichier"`
	`git add fichier3 fichier4`
	`git commit -m "ajout du fichier 3 et 4"`
	`git log`
	
	`user@S203:~/exemple$ git log`
	`commit be99f4166cf068d32233745e234c032dd82cc12b (HEAD -> master)`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:37 2026 +0100`
	
	`ajout du fichier 3 et 4`
	
	`commit 1cb9411aefaa8fd404589caa304307193154490e`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:10 2026 +0100`
	
	`ajout du second fichier`
	
	`commit ad4e99ad7d5e6aa8dc48a2c5df1662eeec81be1b`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:52:50 2026 +0100`
	
	`ajout du premier fichier`
	
	Le résultat est une liste des commits du plus récent au plus ancien avec leurs identifiants (be99f4166...), l'auteur, la date et le message de commit.

Créez une nouvelle branche, faites des modifications, puis fusionnez (merge) cette branche avec la principale. Expliquez.	
	`user@S203:~/exemple$ git checkout -b exemple`
	`Basculement sur la nouvelle branche 'exemple'`
	`user@S203:~/exemple$ echo "dans une branche" > texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit -m "ajout du texte"`
	`[exemple 1ac7443] ajout du texte`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	`user@S203:~/exemple$ git checkout master`
	`Basculement sur la branche 'master'`
	`user@S203:~/exemple$ git merge exemple`
	`Mise à jour be99f41..1ac7443`
	`Fast-forward`
	`texte | 1 +`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	
	Une branche est une chaine de commits parallèle aux autres branches (ici `master` et `exemple`).
	La fusion (merge) permet de combiner les changements d'une branche dans une autre en rajoutant les commits de la branche source vers la branche cible.

Question	Réponse justifiée / Commandes

Qu'est-ce qu'un conflit de fusion ? Provoquez-en un et montrez comment le résoudre.	`user@S203:~/exemple$ git merge conflit`
	`Fusion automatique de texte`
	`CONFLIT (contenu) : Conflit de fusion dans texte`
	`La fusion automatique a échoué ; réglez les conflits et validez le résultat.`
	`user@S203:~/exemple$ nano texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit`
	`[master 8591824] Merge branch 'conflit'`
	
	Un conflit survient lorsque deux branches modifient le même fichier de façon différente.
	Lors de la fusion, Git met le processus en pause et demande une résolution manuelle (choisir le code à garder et supprimer les marqueurs de conflit).
	Pour résoudre le conflit, on modifie le fichier puis on le `git add` pour le marquer comme résolu, avant de faire un commit de fusion.
	
	`<<<<<<< HEAD`
	`conflit 1`
	`=======`
	`conflit 2`
	`>>>>>>> conflit`

Utilisez git diff pour comparer deux commits. Expliquez la sortie.	`diff --git a/texte b/texte`
	`index 9d600a1..7255def 100644`
	`--- a/texte`
	`+++ b/texte`
	`@@ -1 +1 @@`
	`-dans une branche`
	`+conflit 1`
	
	Les premières lignes indiquent les fichiers comparés (`a/texte` et `b/texte`) et les méta-données de la comparaison. (`index 9d600a1..7255def`)
	Les lignes précédées d'un `-` en rouge sont des suppressions et celles avec un `+` en vert sont des ajouts.
	Les autres lignes affichées servent de contexte autour du code modifié.

Qu'est-ce que le fichier .gitignore ? Créez-en un pour ignorer les fichiers Java (.class, .jar).	C'est un fichier texte permettant de lister les motifs des fichiers/dossiers qui ne doivent pas être suivis et versionnés par Git.
	
	`*.class`
	`*.jar`

[1] Page de manuel usermod

[2] Page de manuel mount

[3] Wikipedia Processeur 64 bits

[4] Page de manuel IP

[5] Page de manuel groups

[6] Page de manuel id

[7] Page de manuel su

[8] Page de manuel sudo

[9] Page de manuel uname

[10] Documentation git config

[11] Le mode intelligent est un serveur http qui comprend l'authentification et les push, tandis que le mode idiot est juste un serveur http basique qui distribue le dossier `.git`.

[12] Livre Git - Les Protocoles

[13] Documentation git config

[14] Le mode intelligent est un serveur http qui comprend l'authentification et les push, tandis que le mode idiot est juste un serveur http basique qui distribue le dossier `.git`.

[15] Livre Git - Les Protocoles